Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Full Stack development MERN Practical
Practical– IX

Roll No.: T009	Name: Siddhesh Kamble
Class: TYIT	Batch: 01
Date of Assignment: : 30/09/2026	Date/Time of Submission: 30/09/2026

Practical 9: User Data Management

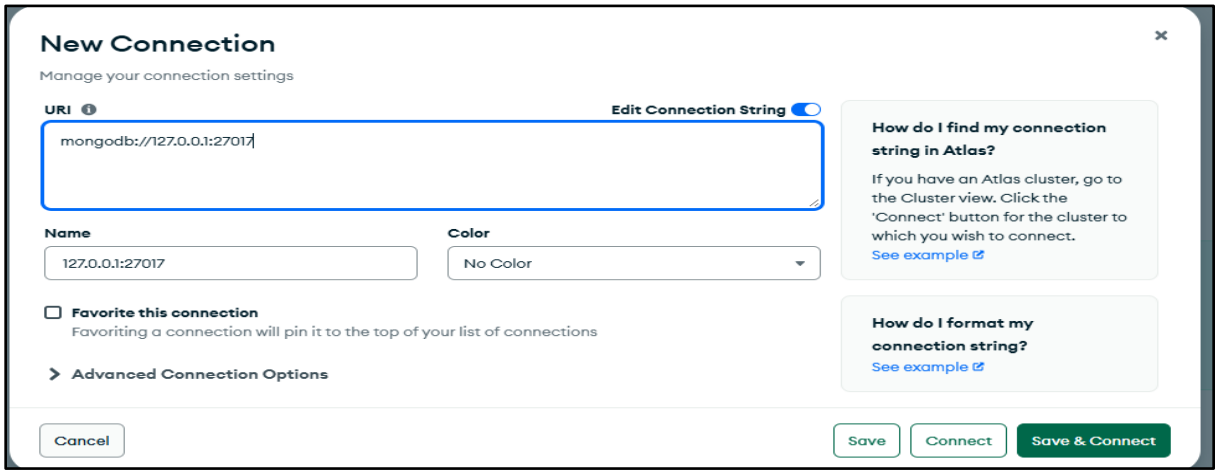
Aim

To create a User Data Management application using **Express, MongoDB and React** for adding, retrieving, updating and deleting user records.

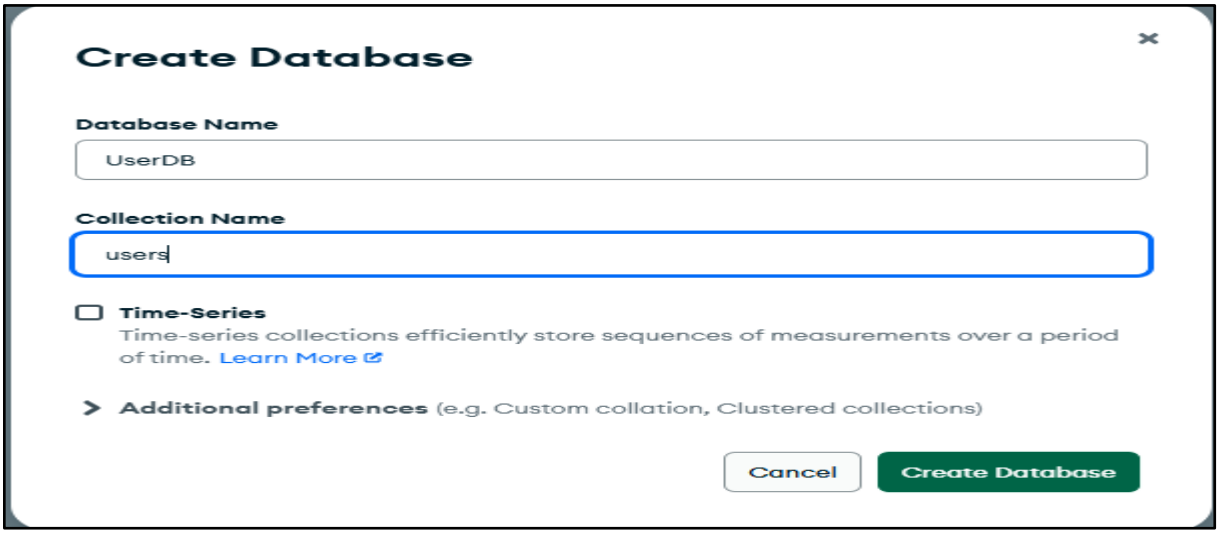
Part A: Create MongoDB Database

Steps:

1. Open MongoDB Compass.
2. Connect to mongodb://127.0.0.1:27017.



3. Create database UserDB.



4. Create collection users.
5. Insert sample user records.



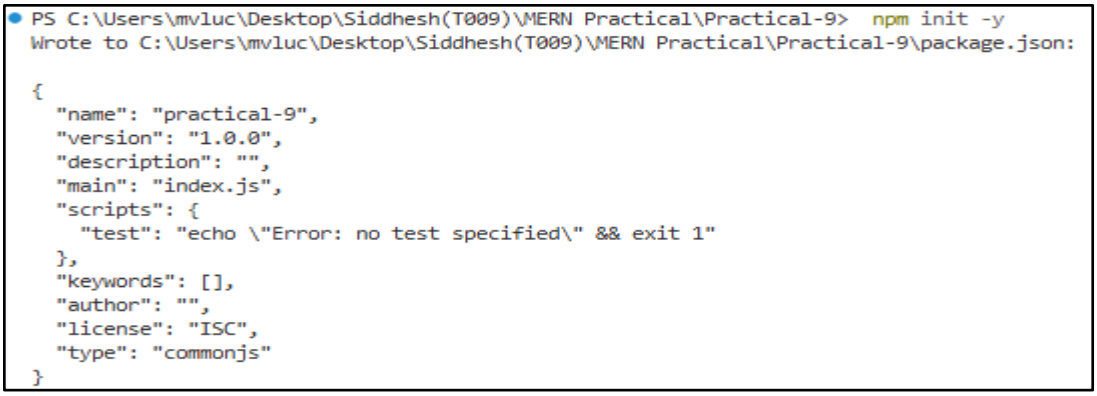
6. Use find() to display the records.



Part B: Create Backend

Steps:

1. Create a folder named Practical-9.
2. Open the folder in VS Code.
3. Run npm init -y.



4. Install mongoose, express and cors.

```
PS C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\Practical-9> npm install mongoose express cors

added 88 packages, and audited 89 packages in 4s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

5. Create server.js.

server.js :

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
const PORT = 3000;
// Middleware
app.use(cors());
app.use(express.json());
// MongoDB Connection
mongoose.connect("mongodb://127.0.0.1:27017/UserDB")
.then(() => {
    console.log("MongoDB Connected Successfully");
})
.catch((error) => {
    console.log("MongoDB Connection Failed");
    console.log(error);
});
// GET - Retrieve Users
app.get("/users", async (req, res) => {
    try {
        const users = await mongoose.connection.db
            .collection("users")
            .find()
            .toArray();

        res.json(users);
    }
    catch (error) {
        res.status(500).json({
            message: "Error fetching users"
        });
    }
});
// POST - Add User
app.post("/users", async (req, res) => {
    try {
        const user = req.body;
        await mongoose.connection.db
            .collection("users")
            .insertOne(user);
        res.json({
            message: "User added successfully"
        });
    }
    catch (error) {
        res.status(500).json({
            message: "Error adding user"
        });
    }
});
// PUT - Update User
app.put("/users/:id", async (req, res) => {
    try {
        const id = new mongoose.Types.ObjectId(req.params.id);
        const updatedUser = req.body;

        await mongoose.connection.db
```

```
                .collection("users")
                .updateOne(
                    { _id: id },
                    {
                        $set: {
                            name: updatedUser.name,
                            email: updatedUser.email,
                            city: updatedUser.city
                        }
                    }
                );

            res.json({
                message: "User updated successfully"
            });
        }
        catch (error) {
            res.status(500).json({
                message: "Error updating user"
            });
        }
    });
    // DELETE - Delete User
    app.delete("/users/:id", async (req, res) => {
        try {
            const id = new mongoose.Types.ObjectId(req.params.id);

            await mongoose.connection.db
                .collection("users")
                .deleteOne({
                    _id: id
                });

            res.json({
                message: "User deleted successfully"
            });
        }
        catch (error) {
            res.status(500).json({
                message: "Error deleting user"
            });
        }
    });
    // Start Server
    app.listen(PORT, () => {
        console.log(`Server is running on http://localhost:${PORT}`);
    });
```

6. Connect Express with MongoDB UserDB.
7. Create GET, POST, PUT and DELETE APIs.
8. Start the server using node server.js.

```
PS C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\Practical-9> node server.js
Server is running on http://localhost:3000
MongoDB Connected Successfully
```

Part C: Create Frontend

Steps:

1. Create a React project using Vite.

```
PS C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\Practical-9> npm create vite@latest frontend

> practical-9@1.0.0 npx
> create-vite frontend

|
◇  Select a framework:
|  React
|
◇  Select a variant:
|  JavaScript
|
◇  Which linter to use?
|  ESLint
|
◇  Install with npm and start now?
|  Yes
|
◇  Scaffolding project in C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\Practical-9\frontend...
|
◇  Installing dependencies with npm...

added 143 packages, and audited 144 packages in 15s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
◇  Starting dev server...

> frontend@0.0.0 dev
> vite


  VITE v8.3.1  ready in 2933 ms

  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
```

```
PS C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\Practical-9\frontend> npm install

up to date, audited 144 packages in 1s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

2. Install Axios.

```
PS C:\Users\mvluc\Desktop\Siddhesh(T009)\MERN Practical\Practical-9\frontend> npm install axios

added 25 packages, and audited 169 packages in 1s

37 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

3. Open src/App.jsx.
4. Create a form for entering user details.
5. Use Axios to communicate with the Express API.
6. Display users in a table.
7. Add Edit and Delete buttons.

App.jsx :

```jsx
import { useEffect, useState } from "react";
import axios from "axios";
function App() {
  // Store all users
```

```
const [users, setUsers] = useState([]);
// Form data
const [name, setName] = useState("");
const [email, setEmail] = useState("");
const [city, setCity] = useState("");
// Store ID of user being edited
const [editId, setEditId] = useState(null);
// Get Users from MongoDB
const getUsers = () => {
  axios.get("http://localhost:3000/users")
    .then((response) => {
      setUsers(response.data);
    })
    .catch((error) => {
      console.log(error);
    });
};
// Load users when page starts
useEffect(() => {
  getUsers();
}, []);
// Add or Update User
const saveUser = () => {
  const user = {
    name: name,
    email: email,
    city: city
  };
  // UPDATE existing user
  if (editId) {
    axios.put(`http://localhost:3000/users/${editId}`, user)
      .then((response) => {
        alert(response.data.message);
        clearForm();
        getUsers();
      })
      .catch((error) => {
        console.log(error);
      });
  }
  // ADD new user
  else {
    axios.post("http://localhost:3000/users", user)
      .then((response) => {
        alert(response.data.message);
        clearForm();
        getUsers();
      })
      .catch((error) => {
        console.log(error);
      });
  }
};
// Edit User
const editUser = (user) => {
  setEditId(user._id);
  setName(user.name);
  setEmail(user.email);
  setCity(user.city);
};
// Delete User
const deleteUser = (id) => {
  if (window.confirm("Are you sure you want to delete this user?")) {
    axios.delete(`http://localhost:3000/users/${id}`)
      .then((response) => {
```

```
            alert(response.data.message);
            getUsers();
          })
          .catch((error) => {
            console.log(error);
          });
      }
    };
    // Clear Form
    const clearForm = () => {
      setName("");
      setEmail("");
      setCity("");
      setEditId(null);
    };
    return (
      <div>
        <h1>User Management System</h1>
        {/* User Form */}
        <h2>
          {editId ? "Update User" : "Add User"}
        </h2>
        <input
          type="text"
          placeholder="Enter Name"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
        <br /><br />
        <input
          type="email"
          placeholder="Enter Email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
        <br /><br />
        <input
          type="text"
          placeholder="Enter City"
          value={city}
          onChange={(e) => setCity(e.target.value)}
        />
        <br /><br />
        <button onClick={saveUser}>
          {editId ? "Update User" : "Add User"}
        </button>
        {editId && (
          <button onClick={clearForm}>
            Cancel
          </button>
        )}
        <hr />
        {/* Display Users */}
        <h2>User List</h2>
        <table border="1">
          <thead>
            <tr>
              <th>Name</th>
              <th>Email</th>
              <th>City</th>
              <th>Actions</th>
            </tr>
          </thead>
          <tbody>
            {users.map((user) => (
```

```
                <tr key={user._id}>
                  <td>{user.name}</td>
                  <td>{user.email}</td>
                  <td>{user.city}</td>
                  <td>
                    <button onClick={() => editUser(user)}>
                      Edit
                    </button>
                    <button onClick={() => deleteUser(user._id)}>
                      Delete
                    </button>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      );
    }
    export default App;
```

Part D: Execution

Backend:

node server.js

Frontend:

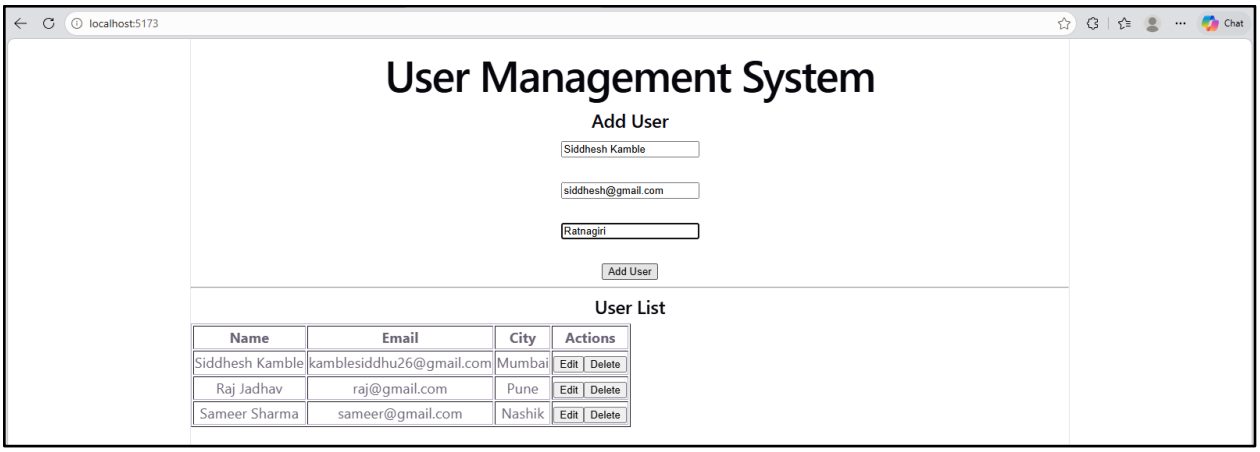
npm run dev

The backend and frontend are executed in **two separate terminals**.



CRUD Operations

Operation	HTTP Method	API
Retrieve users	GET	/users
Add user	POST	/users
Update user	PUT	/users/:id
Delete user	DELETE	/users/:id



Output

The React application displays a **User Management System** containing a user form and user table. Users can be **added, viewed, edited and deleted**, and the updated records are displayed dynamically.

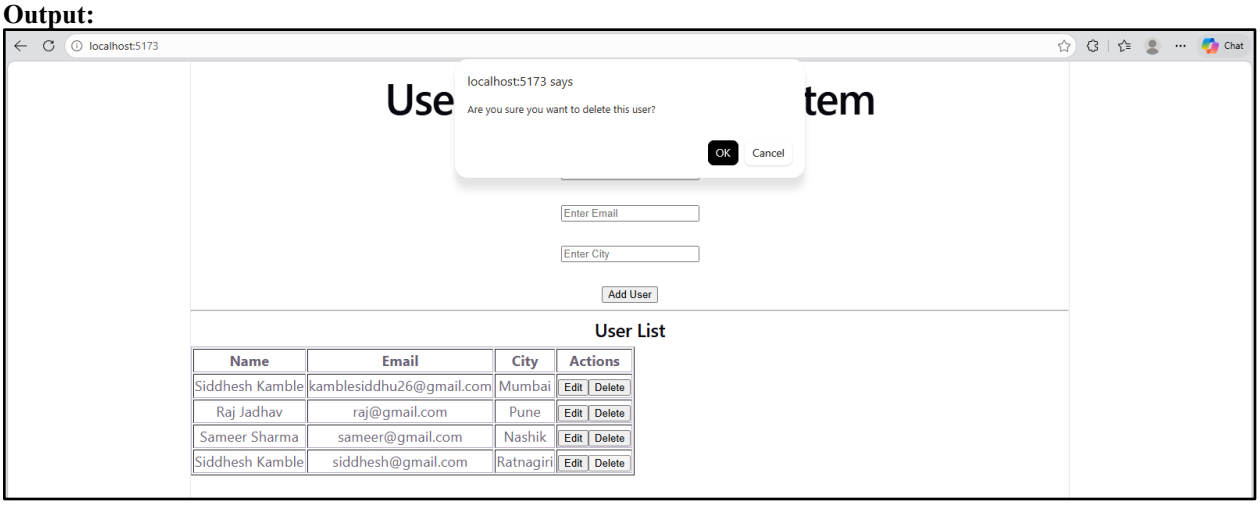
---

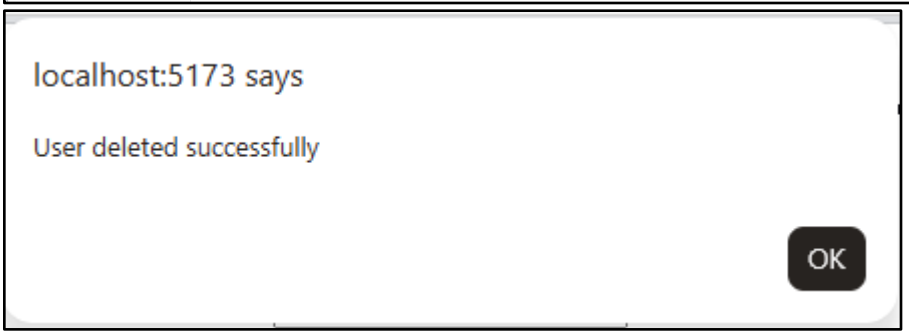
e. Implement Delete Operation

Steps

1. Click the **Delete** button for a selected user.
2. Confirm the delete operation.
3. Axios sends a DELETE request to the Express server.
4. The selected user is deleted from MongoDB.
5. The user is removed from the list.

Output:





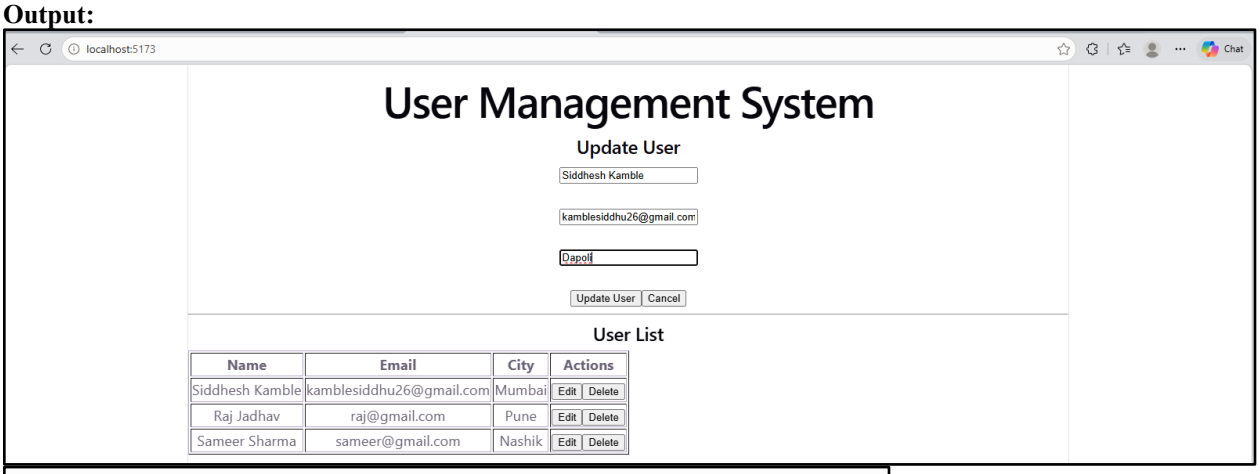
---

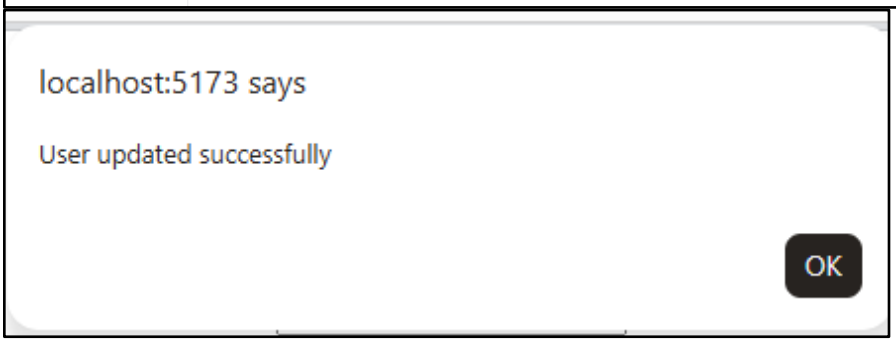
f. Display Updated Records Dynamically in React

Steps

1. After adding, updating, or deleting a user, call getUsers().
2. Fetch the latest records from MongoDB using the GET API.
3. Update the React state using setUsers().
4. The user table automatically displays the updated records without refreshing the page.

Output:





Result

The User Data Management application was successfully created using **React, Express, MongoDB and Axios**, implementing complete CRUD operations.